

Table 2. Summary of DMS assays used.

DMS	Description	Function Tested	Single Mutants	Multiple Mutants
A4_HUMAN_Seuma (Seuma et al., 2022)	Amyloid-beta peptide	Aggregation	796	14,015
AMFR_HUMAN_Tsuboyama (Tsuboyama et al., 2023)	E3 ubiquitin-protein ligase AMFR	Stability	820	2,152
BBC1_YEAST_Tsuboyama (Tsuboyama et al., 2023)	Myosin tail region-interacting protein MTI1	Stability	1,084	985
CAPSD_AAV2S_Sinai (Sinai et al., 2021)	Adeno-associated virus capsid	Viral production	532	41,786
DLG4_HUMAN_Faure (Faure et al., 2022)	Third PDZ domain of PSD95	Yeast growth	1,280	5,696
F7YBW8_MESOW_Ding (Ding et al., 2024)	Antitoxin ParD3	Growth enrichment	80	7,842
GFP_AEQVL_Sarkisyan (Sarkisyan et al., 2016)	Green fluorescent protein	Fluorescence	1,084	50,630
GRB2_HUMAN_Faure (Faure et al., 2022)	C-terminal SH3 domain of GRB2	Yeast growth	1,034	62,232
HIS7_YEAST_Pokusaeva (Pokusaeva et al., 2019)	IGP dehydratase (HIS3)	Growth	168	495,969
RASK_HUMAN_Weng_abundance (Weng et al., 2023)	KRAS	Yeast growth	3,066	22,946
SPG1_STRSG_Olson (Olson et al., 2014)	IgG-binding domain of protein G	Binding	1,045	535,917
YAP1_HUMAN_Araya (Araya et al., 2012)	hYAP65 WW domain	Peptide binding	362	9,713

D.2. Function Prediction Circuit Discovery

Data. To identify function circuits, we use 12 DMS assays from ProteinGym (Notin et al., 2023a). We selected these proteins to ensure robust evaluation across a wide variety of functions (Table 2). Additionally, all of these DMS assays contain multiple mutants, which we utilize for our steering experiments. For each DMS, we collected their MLP outputs from layer L without mean-pooling across all sequences.

Training Splits. We follow the supervised ProteinGym benchmark to determine our training data. The supervised benchmark is divided into two groupings: single and multiple mutations. In the single grouping, we train our CNN probes on single mutants and test on single mutants. ProteinGym uses three distinct cross-validation schemes to assess the ability of the CNN probe. In the *random* scheme, each mutation is randomly assigned to one of five folds. In the *contiguous* scheme, the sequence is split up into five continuous segments along its length. Mutations are assigned to each fold depending on which segment their position in the sequence falls under. Lastly, in the *modulo* scheme, the modulo operator is used to assign mutated positions to each fold. For example, position one is assigned to fold one, position two is assigned to fold two, continuing until position five is assigned to fold five. Position six is then looped back to fold one, position seven is looped back to fold two, and so on and so forth until the end of the sequence. In the multiple grouping, ProteinGym utilizes only the random scheme, since multiple mutations can exist outside the folds defined by the contiguous and modulo schemes. We train our CNN probes using five-fold cross validation, reporting the Spearman correlation each time. Three of the folds are reserved for training and use the validation fold to monitor loss for early stopping. Similar to the family circuit setup, the validation fold is also used for circuit discovery and computing attribution scores. We compute all Spearman correlations based on the test set.

CNN Probe Training. Following a similar methodology to (Notin et al., 2023b), the CNN architecture is set as a 1D convolutional layer with a kernel size of 7 and same padding, followed by a ReLU activation and dropout with a probability of 0.1. The output is then passed through a linear head to produce a scalar fitness prediction. We train the probe using the AdamW optimizer with a maximum learning rate of 3×10^{-4} and a cosine annealing schedule with a linear warmup of 100 steps. Training proceeds for 10,000 steps with a batch size of 128, minimizing the Mean Squared Error (MSE) loss. We employ early stopping with a patience of 10 evaluation steps based on the validation loss.

Circuit Discovery. Similar to the family circuit setup, we define m_{clean} to be the Spearman correlation of the original CNN probe. We then compute m_{all} as the Spearman correlation with all the latents active, and set $\theta = \min(0.7 \times m_{\text{clean}}, m_{\text{all}})$ to be the comparable Spearman correlation of the circuit. For each latent, we sum the attribution scores across all functional sequences (denoted by the entry `DMS_score_bin = 1` per mutant) to get a ranking of the top latents, and then proceed with our iterative greedy search method as previously described.

Plotting. To create Fig. 1d, Fig. 5b, and Fig. 7, we only include the results where the performance of ESM2 obtains a Spearman correlation of above or equal to 0.01, for similar reasons as in the protein family case. These circuits are used to plot the distribution of latents in Fig. 6b. We leave all function prediction circuits on our HuggingFace.

D.3. Implementation of Circuit Steering

Data and Circuits. We conduct our steering experiments on a subset of DMS assays from Table 2: SPG1_STRSG_Olson_2014, HIS7_YEAST_Pokusaeva_2019, GRB2_HUMAN_Faure_2021, GFP_AEQVL_Sarkisyan_2016, CAPSD_AAV2S_Sinai_2021, RASK_HUMAN_Weng_2022_abundance, and A4_HUMAN_Seuma_2022. We picked these

DMS assays due to having more than 10,000 multiple mutants present, which provides more training data for our CNN evaluation model to better learn the fitness landscape. For each of the replacement models, we steer the circuits attained from the multiple mutation groupings.

CNN Evaluation Model Training. To create a CNN evaluation model to approximate the fitness landscape, we copy the CNN architecture, hyperparameters, and training strategy from our function prediction circuit CNN probe training methodology. We randomly split the entire DMS assay into a training (90%) and test (10%) set.

Attribution-Weighted Sampling. Similar to the function circuit discovery procedure, we rank all latents by descending attribution score. To select a diverse set of influential features, we convert the attribution scores of all circuit latents into a probability distribution. We then sample C many latents without replacement from this distribution for five trials per cross-validation fold. In our experiments, we steer across $C \in [4, 8, 16]$ many latents.

Steering. For each DMS assay, we first feed in the wildtype sequence into ESM2. At each token index, we find the feature activations $\mathbf{a}^\ell$. If a_i^ℓ is in the set of latents to be steered, we clamp a_i^ℓ to α times its maximum activation value across all tokens and all $i \in [d_{\text{latent}}]$.

We then decode the latents using the steered version of the activations $\mathbf{a}^\ell$ to obtain the steered reconstruction. To ensure that the model output is affected only by the steering vector and not by the lossy reconstruction error of the replacement model, we apply an error correction term. We first pass the unsteered wildtype sequence through the replacement model to generate a reconstruction, and calculate the difference between the reconstruction and the true ESM2 output. This residual error is then added to the steered output. From here, we continue the forward pass as specified by each replacement model to output the logits.

To determine which amino acids to mutate, we compare the logits of the steered sequence to the logits of the wildtype sequence. Following (Tsui et al., 2025b), at each amino acid position, we compute the cosine similarity between the respective logit vectors. We mutate a position if the cosine similarity is below 0.98, which ensures we only mutate amino acids where ESM2 has made a meaningful change. Additionally, to ensure that our CNN evaluation model is a good proxy for fitness, we constrain our search space to designing protein variants with a maximum of five mutations away. In cases where a steered sequence suggests more than five mutations, we select five mutations based on the largest logit values and set the rest of the sequence to the wildtype.

Experimental Setup. For each C , we steer our latents over values of α from 0.1 to 5 over 25 evenly spaced steps. For each multiplier, we use our CNN probes from circuit discovery to score each mutated sequence. We then select the top 50 mutated sequences with the highest predicted fitness and score them using our CNN evaluation model. We perform this over all five cross-validation folds and over five trials per cross-validation fold.

To create the random baseline, for each of the steered sequences, we construct a random sequence that has the same number of mutations as the steered sequence, with the positions and mutations chosen uniformly at random. We then randomly select 50 of these sequences and score them using the CNN evaluation model.

D.4. Implementation of CAA Steering

We add Contrastive Activation Addition (CAA) as a baseline to our steering experiments due to its ability to steer at all layers (as in the case with ProtoMech and PLT). We adapt the CAA (Rimsky et al., 2024) steering method from (Huang et al., 2025), but modify the algorithm slightly to generate mutations locally around a wildtype sequence. Starting with a given DMS dataset, we divide the dataset into positive and negative sequences based on a threshold. We then run 10 trials, where for each trial we sample 10% of the positive and negative sequences randomly to create positive set $\mathcal{P}$ and negative set $\mathcal{N}$. Following the notation and formulation in (Huang et al., 2025), we generate the steering vector $\mathbf{v}_\ell$ as:

$$\mathbf{v}_\ell = \frac{1}{|\mathcal{P}|} \sum_{x_p \in \mathcal{P}} \mathbf{h}_\ell^{\text{avg}}(x_p) - \frac{1}{|\mathcal{N}|} \sum_{x_n \in \mathcal{N}} \mathbf{h}_\ell^{\text{avg}}(x_n), \quad (10)$$

where $\mathbf{h}_\ell$ represents the mean-pooled ESM activation at layer ℓ ($\mathbf{x}^\ell + \mathbf{y}^\ell$) and $x_p \in \mathcal{P}$ and $x_n \in \mathcal{N}$ represent a sequence from the positive and negative set, respectively. We then add the steering vector as:

$$\tilde{\mathbf{h}}_\ell = \mathbf{h}_\ell + \alpha \mathbf{v}_\ell \quad (11)$$

and re-normalize $\tilde{\mathbf{h}}_\ell$ to have the same norm as $\mathbf{h}_\ell$. The addition in Equation (11) is done at each token and each layer. We do this at every layer and decode to the logits. To choose mutations by changing to the amino acid token at each position to the

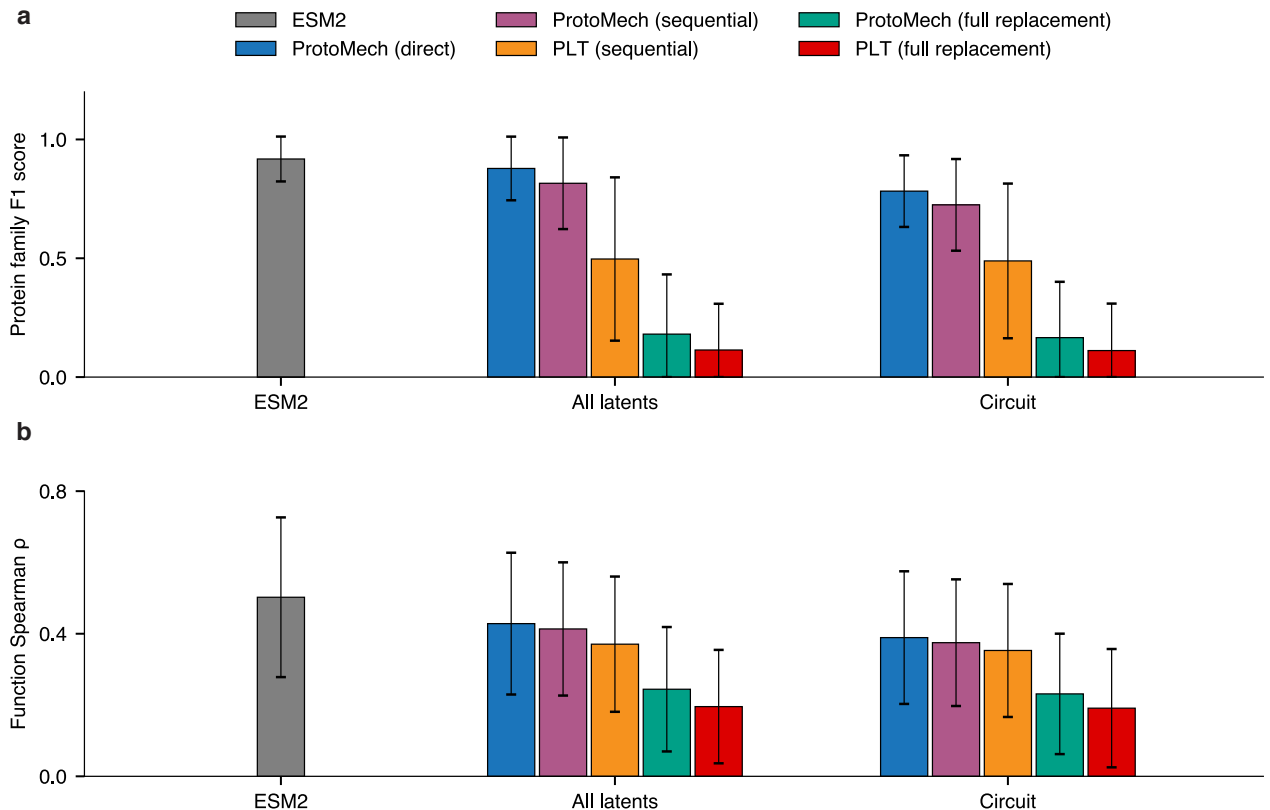


Figure 5. Performance of all replacement models on **a**, protein family classification and **b**, function prediction tasks.

maximum logit probability. In cases where the mutated sequence would be over 5 mutations, we choose the 5 mutations with the highest logit probabilities. We note that we did not use the cosine similarity trick as in steering, as we experienced difficulties generating diverse sequences (see below) and wanted to maximize the amount of sequences generated. We run CAA twice with ten trials each. We first set the positive set to be the set of sequences with a score in the top or bottom 10%, and we then set the positive sequences as functioning sequences (indicated in ProteinGym by a `DMS_score_bin` of 1) and the negative sequences as nonfunctioning sequences (indicated in ProteinGym by a `DMS_score_bin` of 0). We steer over values of α from 0.1 to 5 over 25 evenly spaced steps.

When running CAA, we observe that a weakness of this method is a lack of diverse sequences generated. We notice that despite steering over multiple trials, CAA fails to produce many unique sequences for some DMS datasets (and in some cases produces only one unique sequence), unlike other methods. Therefore, when reporting our steering results, in cases where the top 10% and 20% score happens to only contain a single sequence (which happens when the rest of the sequences' evaluation scores are significantly worse than the max sequence score), we take the top 10% and 20% sequences, ranked by their score.

E. Additional Experimental Results

E.1. Circuit Discovery

As noted in Appendix C.2, in the main text (Fig. 1c), we compare the performance of ProtoMech and PLT using the *sequential* replacement model on circuit discovery tasks to ensure a fair comparison of the representational power of both architectures. In this section, we expand our evaluation to include the full spectrum of replacement models across both protein family classification and function prediction tasks (Fig. 5).

We observe that the ProtoMech direct replacement model achieves the highest performance among all other replacement models. We attribute this to the direct replacement model's ability to bypass layer-wise sequential operations, avoiding